# УЧЕБНИК ПО GIT

Контроль версий для веб-разработчика

Этот учебник — практическое введение в Git для веб-разработчика-новичка. Git — самая популярная система контроля версий в мире: без неё не обходится ни один профессиональный проект. Учебник дополняет серию из семи предыдущих (HTML, CSS, SVG, PHP, SQLite+PHP, Хостинг, Цикл разработки) и показывает, как использовать Git в связке с ними.

Все примеры рассчитаны на запуск в терминале macOS, Linux или Windows (Git Bash / WSL). Команды одинаково работают в любой ОС — Git кроссплатформенный.

---

## ОГЛАВЛЕНИЕ


- Глава 1. Что такое Git
- Глава 2. Базовый цикл: init, add, commit
- Глава 3. Просмотр истории и изменений
- Глава 4. Ветки
- Глава 5. Слияния и конфликты
- Глава 6. Rebase
- Глава 7. Удалённые репозитории
- Глава 8. .gitignore, stash, cherry-pick, tags
- Глава 9. Конвенции и workflow
- Глава 10. Git-деплой
- Глава 11. Аварийное восстановление
- Глава 12. GitHub-фичи, задачи и итоги


---

## ГЛАВА 1. ЧТО ТАКОЕ GIT

## 1.1. Зачем нужен контроль версий

Представьте разработку без Git. У вас папка с проектом, вы правите index.php, потом передумали и хотите вернуться. Без истории изменений — только память, что было раньше. Через месяц не вспомните.

Что обычно делают без Git:

- Копии папок: project_v1, project_v2, project_final, project_final_FINAL
- ZIP-архивы с датами: site_2026-10-26.zip
- Облачные копии: Dropbox, Яндекс.Диск с историей
- Комментарии в коде: // старый вариант, оставил на всякий

Всё это — костыли. Git решает проблему системно: хранит историю, позволяет возвращаться к любому моменту, даёт ветки для экспериментов, синхронизирует работу команды.

## 1.2. Что такое Git

Git — распределённая система контроля версий. Распределённая значит, что у каждого разработчика есть полная копия репозитория со всей историей, не только текущее состояние. Можно работать офлайн, синхронизироваться через любой удалённый сервер.

Ключевые понятия:

- Репозиторий (repo) — папка с историей изменений
- Коммит (commit) — фиксация изменений в один момент
- Ветка (branch) — параллельная линия разработки
- Индекс (staging area) — подготовка к коммиту
- Remote — удалённая копия репозитория (на GitHub)

## 1.3. Установка Git

На macOS и Linux Git часто уже установлен. Проверим:

```
git --version
# git version 2.43.0  — например
```

Если не установлен:

```
# Ubuntu/Debian
sudo apt update &amp;amp;&amp;amp; sudo apt install git

# macOS (через Homebrew)
brew install git

# Windows: скачать с git-scm.com
# установщик включает Git Bash
```

## 1.4. Первая настройка

Перед первым коммитом Git должен знать, кто вы. Имя и email записываются в каждый коммит — так в истории видно автора изменения:

```
# Глобальная настройка (один раз)
git config --global user.name  "Иван Петров"
git config --global user.email "ivan@example.com"

# Проверить
git config --global --list

# Настройка только для текущего проекта
# (перекрывает глобальную):
cd ~/projects/mysite
git config user.email "work@mycompany.ru"
```

## 1.5. Полезные глобальные настройки

```
# Имя основной ветки по умолчанию
git config --global init.defaultBranch main

# Цветной вывод
git config --global color.ui auto

# Редактор для коммит-сообщений
git config --global core.editor nano

# Авто-определение окончания строк
git config --global core.autocrlf input  # Mac/Linux
git config --global core.autocrlf true  # Windows

# Кэширование паролей (15 минут)
git config --global credential.helper cache
```

## 1.6. Связка с серией учебников

Этот учебник дополняет предыдущие семь. Связки:

- PHP (главы 3, 8): .gitignore для vendor/, .env

- SQLite+PHP: игнорировать data/*.sqlite, миграции в Git
- Хостинг: SSH-ключи уже настроены, используем для git push
- Цикл разработки (глава 3): тот же workflow, теперь подробно
- HTML/CSS/SVG: исходники в Git, не теряем историю дизайна

---

# ГЛАВА 2. БАЗОВЫЙ ЦИКЛ: INIT, ADD, COMMIT

## 2.1. git init — создать репозиторий

Любую папку можно превратить в Git-репозиторий одной командой:

```
cd ~/projects/mysite
git init

# Вывод:
# Initialized empty Git repository
# in /Users/ivan/projects/mysite/.git/
```

Создаётся скрытая папка .git/ — в ней хранится вся история изменений. Никогда не редактируйте её вручную и не удаляйте, если не хотите потерять историю.

## 2.2. git status — что происходит

Самая частая команда. Показывает текущее состояние репозитория: какие файлы изменились, что в индексе, что не отслеживается.

```
git status

# Вывод для нового репо:
# On branch main
#
# No commits yet
#
# Untracked files:
#   (use "git add <file>..." to include)
#         index.php
#         style.css
#
# nothing added to commit but untracked
# files present (use "git add" to track)
```

Вывод git status — ваш главный ориентир. Привыкайте смотреть его перед каждым коммитом и после каждого действия.

## 2.3. git add — добавить в индекс

Git различает три зоны:

- Рабочая директория — файлы на диске, которые вы правите
- Индекс (staging) — подготовленные к коммиту изменения
- Репозиторий — зафиксированные коммиты

git add переносит изменения из рабочей директории в индекс. Можно добавлять по одному файлу или всё сразу:

```
git add index.php            # один файл
git add index.php style.css # несколько
git add .                    # всё в текущей папке
git add -A                   # все изменения (включая удалённые)
git add -p                   # интерактивно по частям
```

git add -p (patch) — мощная команда: показывает каждое изменение и спрашивает, добавлять ли его. Удобно, когда в одном файле несвязанные правки — можно разбить на разные коммиты.

## 2.4. git commit — зафиксировать

git commit создаёт слепок изменений. После коммита изменения нельзя случайно потерять — они в истории навсегда.

```
git commit -m "feat: добавлена форма обратной связи"

# Вывод:
# [main (root-commit) a3f2c1d]
#  2 files changed, 45 insertions(+)
#  create mode 100644 index.php
#  create mode 100644 style.css
```

Хэш a3f2c1d — уникальный идентификатор коммита. По нему можно обратиться к коммиту в любых командах (git show, git revert и т.д.).

## 2.5. Статусы файлов

Файл в Git может быть в одном из четырёх состояний:

- Untracked — новый, Git его не знает
- Staged — добавлен в индекс, ждёт коммита
- Committed — зафиксирован в истории
- Modified — отслеживается, но изменён после коммита

Цикл жизни файла:

```
Untracked --git add--> Staged --git commit--> Committed
   |                       |
   |                 git add (после правок)
   v                       v
  (не в Git)           Modified
```

### 2.6. Полный первый цикл

Соберём всё вместе — создание первого коммита с нуля:

```
mkdir mysite &amp;amp;&amp;amp; cd mysite
git init
git config user.name  "Иван Петров"
git config user.email "ivan@example.com"

echo "<?php echo 'Hello';" > index.php
echo "body { margin: 0; }" > style.css

git status  # Untracked files: index.php, style.css

git add .
git status  # Changes to be committed:
            #   new file: index.php
            #   new file: style.css

git commit -m "feat: первый коммит, заготовка сайта"

git log --oneline
# a3f2c1d (HEAD -> main)
#   feat: первый коммит, заготовка сайта
```

---

# ГЛАВА 3. ПРОСМОТР ИСТОРИИ И ИЗМЕНЕНИЙ

### 3.1. git log — история

git log показывает все коммиты в обратном хронологическом порядке (сначала новые):

```
git log

# commit a3f2c1d... (HEAD -> main)
# Author: Иван Петров <ivan@example.com>
# Date:   Mon Oct 26 14:32:11 2026
#
#     feat: первый коммит, заготовка сайта
#
```

## 3.2. Полезные опции log

```
# Краткий формат, одна строка на коммит
git log --oneline

# Граф веток + кратко
git log --oneline --graph --all

# Последние 10 коммитов
git log -n 10

# Изменения в каждом коммите
git log -p

# Только изменения конкретного файла
git log -- index.php

# Статистика (+/- по файлам)
git log --stat

# Коммиты конкретного автора
git log --author="Иван"

# За последние 7 дней
git log --since="1 week ago"
```

## 3.3. git show — конкретный коммит

Показывает, что именно сделал выбранный коммит:

```
git show a3f2c1d

# commit a3f2c1d...
# Author: ...
# Date:   ...
#
#     feat: первый коммит
#
# diff --git a/index.php b/index.php
# new file mode 100644
# index 0000000..e69de29
# --- /dev/null
# +++ b/index.php
# @@ -0,0 +1 @@
# +<?php echo 'Hello';
```

## 3.4. git diff — изменения

Показывает разницу между состояниями. Без аргументов — между рабочей директорией и индексом (что не добавлено в staging):

```
git diff

# diff --git a/index.php b/index.php
# index e69de29..8ba32a1 100644
# --- a/index.php
# +++ b/index.php
# @@ -1 +1,2 @@
# -<?php echo 'Hello';
# +<?php
# +echo 'Hello, World!';
```

Разбор вывода:

- --- a/index.php — старая версия
- +++ b/index.php — новая версия
- @@ -1 +1,2 @@ — координаты блока изменений
- - (минус) — удалённые строки
- + (плюс) — добавленные строки

## 3.5. git diff с опциями

```
# Что в индексе, но не закоммичено
git diff --staged
git diff --cached  # синоним

# Сравнить две ветки
git diff main feature/login

# Сравнить два коммита
git diff a3f2c1d 8b2e9f0

# Только имена изменённых файлов
git diff --name-only

# Количество строк (статистика)
git diff --stat
```

---

# ГЛАВА 4. ВЕТКИ

## 4.1. Зачем ветки

Ветка — это параллельная линия разработки. На главной ветке (main) у вас рабочий код. В отдельной ветке можно экспериментировать, не ломая main. Если эксперимент удался — вливаете ветку обратно.

Типичные сценарии веток:

- Новая фича: feature/search
- Исправление бага: fix/login-error
- Срочный хотфикс: hotfix/security-patch
- Релиз: release/v1.2.0
- Личная ветка: ivan/experiment

## 4.2. Просмотр и создание веток

```
# Список локальных веток
git branch
# * main
#   feature/search

# Все ветки, включая удалённые
git branch -a

# Создать новую ветку (остаться в текущей)
git branch feature/search

# Создать и переключиться
git switch -c feature/search
git checkout -b feature/search  # старый синтаксис

# Переключиться на существующую
git switch main
git checkout main  # старый синтаксис
```

git switch появился в Git 2.23 (2019) как более понятная альтернатива git checkout. checkout делает слишком много: и переключает ветки, и откатывает файлы, и создаёт ветки. switch отвечает только за ветки, restore — за откат файлов.

## 4.3. Удаление веток

```
# Безопасное удаление (откажет, если не слита)
git branch -d feature/search

# Принудительное удаление
git branch -D feature/search

# Удалить удалённую ветку
git push origin --delete feature/search
```

## 4.4. Наименование веток

Хорошая конвенция делает историю читаемой. Распространённые префиксы:

- feature/ — новая функциональность
- fix/ — исправление бага
- hotfix/ — срочный патч в продакшен
- release/ — подготовка релиза
- docs/ — только документация
- refactor/ — переработка без новой логики
- chore/ — обслуживание, зависимости

```
git switch -c feature/user-registration
git switch -c fix/contact-form-validation
git switch -c hotfix/sql-injection
```

---

# ГЛАВА 5. СЛИЯНИЯ И КОНФЛИКТЫ

## 5.1. git merge — слияние веток

Когда фича готова, её нужно слить обратно в main. Переключаемся на main и вызываем merge:

```
git switch main
git merge feature/search

# Успешное слияние (fast-forward):
# Updating a3f2c1d..b8e4f02
# Fast-forward
#  search.php | 45 ++++++++++
#  1 file changed, 45 insertions(+)
```

## 5.2. Fast-forward vs no-ff merge

Fast-forward (по умолчанию) — если main не двигался, Git просто перемещает указатель main вперёд. История линейная, как будто ветки и не было.

Иногда хочется сохранить факт ветвления в истории (для понимания, что была фича). Тогда используют --no-ff:

```
git merge --no-ff feature/search

# Создаёт merge commit — отдельный
# коммит "Merge branch 'feature/search'"
```

В истории будет видно:

```
*   a1b2c3d (HEAD -> main)
# |\  Merge branch 'feature/search'
# | * b8e4f02  добавлен поиск
# | * 9c1d7e8  индекс для поиска
# |/
# * a3f2c1d  предыдущий коммит main
```

## 5.3. Конфликты слияния

Конфликт возникает, когда обе ветки изменили одни и те же строки файла. Git не знает, какую версию выбрать, и просит вас разобраться.

```
git merge feature/search

# Auto-merging index.php
# CONFLICT (content): Merge conflict
# in index.php
# Automatic merge failed; fix conflicts
# and commit the result.
```

## 5.4. Что выглядит в файле

Git вставляет в файл маркеры конфликта:

```
<?php
<<<<<<< HEAD
$title = 'Главная страница';
=======
$title = 'Добро пожаловать';
>>>>>>> feature/search

echo $title;
```

Разбор:

- <<<<<<< HEAD — версия из текущей ветки (main)
- ======= — разделитель
- >>>>>>> feature/search — версия из вливаемой ветки

Нужно отредактировать файл, оставив правильную версию (или комбинацию обеих), и удалить маркеры:

```
<?php
$title = 'Добро пожаловать на главную';

echo $title;
```

После разрешения конфликта:

```
git add index.php
git commit  # без -m откроется редактор
            # с сообщением "Merge branch ..."
```

### 5.5. Отмена слияния

Если запутались в конфликте и хотите начать заново:

```
git merge --abort

# Возвращает состояние до начала merge
```

### 5.6. Стратегии разрешения

Когда не хочется разбираться вручную:

```
# Принять версию из текущей ветки (наш вариант)
git checkout --ours index.php

# Принять версию из вливаемой ветки (их вариант)
git checkout --theirs index.php

# После выбора:
git add index.php
git commit
```

---

# ГЛАВА 6. REBASE

### 6.1. Что такое rebase

git rebase — альтернатива merge. Вместо создания merge-коммита, rebase берёт коммиты из вашей ветки и «пересаживает» их на актуальный конец main. История становится линейной, без ветвлений.

Пример: вы делали фичу feature/x, пока в main появились новые коммиты. Чтобы получить эти обновления в своей ветке:

```
git switch feature/x
git rebase main

# Git по очереди:
# 1. Откатывает коммиты feature/x
# 2. Применяет новые коммиты main
# 3. Применяет ваши коммиты поверх
```

### 6.2. Merge vs Rebase

Главное отличие:

- merge: сохраняет историю как есть, создаёт merge commit
- rebase: переписывает историю, делает её линейной

Когда что использовать:

- merge — для публичных веток, где важна история
- merge --no-ff — для фич, чтобы видеть merge point
- rebase — для локальных коммитов перед отправкой
- rebase -i — для чистки истории перед push

## 6.3. Золотое правило rebase

НИКОГДА не делайте rebase публичных веток (main, develop, чужих feature-веток). Rebase переписывает историю, и у других разработчиков их локальные копии станут несовместимыми.

Безопасно rebase'ить:

- Свою локальную ветку, не отправленную в общий репо
- Feature-ветку, которую вы не делили с другими

## 6.4. Интерактивный rebase

git rebase -i — мощный инструмент для чистки истории перед отправкой. Позволяет объединять, переименовывать, переупорядочивать, удалять коммиты.

```
git rebase -i HEAD~3  # последние 3 коммита

# Откроется редактор:
# pick   a3f2c1d  feat: форма
# pick   b8e4f02  fix: валидация
# pick   9c1d7e8  typo

# Доступные действия:
# pick    — оставить как есть
# reword  — изменить сообщение
# squash  — объединить с предыдущим
# fixup   — squash, но выбросить сообщение
# edit    — остановиться, можно менять коммит
# drop    — удалить коммит
# reorder — менять местами строки
```

Частый сценарий: у вас 5 мелких коммитов «fix typo», «ещё фикс», «wip». Перед push хочется собрать в один чистый:

```
# Было:
# pick  feat: форма
# pick  fix: typo
# pick  wip
# pick  fix: validation

# Меняем на:
# pick    feat: форма
# fixup   fix: typo
# fixup   wip
# reword  fix: validation → fix: валидация email
```

### 6.5. Reflog — страховка

Если rebase пошёл не так, история не потеряна. Команда git reflog показывает все перемещения HEAD — даже те, что были переписаны:

```
git reflog

# a3f2c1d HEAD@{0}: rebase: checkout main
# b8e4f02 HEAD@{1}: rebase: последний коммит
# 9c1d7e8 HEAD@{2}: commit: feat: форма
# ... исходная история до rebase

# Вернуться к состоянию до rebase:
git reset --hard HEAD@{2}
```

---

# ГЛАВА 7. УДАЛЁННЫЕ РЕПОЗИТОРИИ

### 7.1. Что такое remote

Remote — это копия вашего репозитория на другом компьютере, обычно на сервере GitHub, GitLab или Bitbucket. Удалённый репо служит бэкапом и точкой синхронизации.

Основные удалённые хостинги:

- GitHub.com — Microsoft, самый популярный, открытые репо бесплатно
- GitLab.com — open-source, можно self-host
- Bitbucket.org — Atlassian, интеграция с Jira
- Codeberg.org — некоммерческий, для опенсорса
- Свой сервер — Gitolite, Gitea, Forgejo

## 7.2. git clone — копия существующего

Если репозиторий уже есть на GitHub, его клонируют локально:

```
# HTTPS
git clone https://github.com/user/repo.git

# SSH (рекомендуется)
git clone git@github.com:user/repo.git

# В определённую папку
git clone git@github.com:user/repo.git my-folder

# Только последняя версия (для больших репо)
git clone --depth 1 git@github.com:user/repo.git
```

## 7.3. git remote add — связать локальный с удалённым

Если вы создали репо локально (git init), его нужно связать с удалённым:

```
cd ~/projects/mysite
git init

# Добавить remote с именем origin
git remote add origin \
    git@github.com:ivan/mysite.git

# Проверить
git remote -v
# origin  git@github.com:ivan/mysite.git (fetch)
# origin  git@github.com:ivan/mysite.git (push)

# Первая отправка
git push -u origin main
# -u устанавливает upstream: потом можно просто git push
```

## 7.4. git push и git pull

push — отправить локальные коммиты на удалённый сервер. pull — получить удалённые изменения и слить с локальными.

```
# Отправить текущую ветку в origin
git push

# Отправить конкретную ветку
git push origin feature/search

# Получить и слить изменения
git pull

# Получить, но не сливать
git fetch
# git fetch только скачивает, не трогает локальную ветку
```

## 7.5. fetch vs pull

Важное различие:

- fetch: скачивает изменения, не меняет рабочие файлы
- pull: fetch + merge (или rebase) сразу

fetch безопаснее — позволяет сначала посмотреть, что прилетело (git log origin/main), и решить, как слить.

## 7.6. SSH-ключи для GitHub

HTTPS требует вводить логин/пароль (или токен) при каждом push. SSH — ключи, после настройки — бесшовно. По учебнику Хостинг:

```
# Создать ключ
ssh-keygen -t ed25519 -C "ivan@example.com"

# Скопировать открытый ключ в буфер
cat ~/.ssh/id_ed25519.pub | pbcopy   # macOS
cat ~/.ssh/id_ed25519.pub | xclip -sel clip  # Linux

# Добавить в GitHub: Settings → SSH and GPG keys
#    → New SSH key → вставить → Add

# Проверить
ssh -T git@github.com
# Hi ivan! You've successfully authenticated,
```

## 7.7. Fork и upstream

Fork — копия чужого репозитория в ваш аккаунт. Часто для контрибуции в open-source:

```
# 1. На GitHub: Fork -> в вашем аккаунте: ivan/repo
# 2. Клонировать свой fork
git clone git@github.com:ivan/repo.git
cd repo

# 3. Добавить оригинал как upstream
git remote add upstream \
    git@github.com:original/repo.git

# 4. Получать обновления из оригинала
git fetch upstream
git switch main
git merge upstream/main  # синхронизировать main

# 5. Делать фичу и отправить в свой fork
git switch -c feature/my-fix
# ... изменения ...
git push origin feature/my-fix

# 6. На GitHub: открыть Pull Request в original/repo
```

---

# ГЛАВА 8. .GITIGNORE, STASH, CHERRY-PICK, TAGS

## 8.1. .gitignore — что не отслеживать

Файл .gitignore указывает, какие файлы Git должен игнорировать. Это обязательно для каждого проекта.

Синтаксис .gitignore:

- file.txt — игнорировать конкретный файл
- *.log — все .log файлы
- node_modules/ — папка целиком
- /secret — только в корне репо
- secret/ — в любой вложенной папке
- !important.log — НЕ игнорировать (исключение)
- **/temp — temp в любой вложенности

## 8.2. .gitignore для серии учебников

Шаблон .gitignore для проектов из предыдущих учебников (PHP + SQLite + веб-разработка):

```
# === СЕКРЕТЫ ===
.env
.env.local
*.pem
*.key

# === БАЗА ДАННЫХ ===
data/*.sqlite
data/*.sqlite-journal
data/*.sqlite-wal
data/*.sqlite-shm

# === PHP ===
vendor/
composer.lock

# === NODE (если есть фронтенд сборка) ===
node_modules/
npm-debug.log*

# === СБОРКИ ===
dist/
build/
*.min.js
*.min.css

# === ЛОГИ И ВРЕМЕННЫЕ ===
*.log
/tmp/
/cache/

# === ОС ===
.DS_Store
Thumbs.db
desktop.ini

# === IDE ===
.idea/
.vscode/
*.swp
*.swo

# === МЕДИА (загруженные пользователями) ===
public/uploads/
public/assets/img/uploads/
```

## 8.3. git stash — отложить изменения

Бывает: вы правите файл, не закончили, а нужно срочно переключиться на другую ветку. git stash сохраняет изменения и очищает рабочую директорию:

```
git stash

# Saved working directory and index state
# WIP on main: a3f2c1d feat: форма

git switch main  # теперь можно переключаться
# ... делаем что-то срочное ...
git switch feature/x

# Вернуть изменения
git stash pop

# Список всех stash
git stash list
# stash@{0}: WIP on main: a3f2c1d
# stash@{1}: WIP on feature/x: b8e4f02

# Применить конкретный
git stash apply stash@{1}

# Удалить stash
git stash drop stash@{0}
git stash clear  # удалить все
```

## 8.4. git cherry-pick — перенос коммита

Если один коммит из другой ветки нужен в текущей (без слияния всей ветки):

```
git switch main
git cherry-pick b8e4f02

# Создаст новый коммит в main
# с теми же изменениями,
# что и b8e4f02 в feature/x
```

Полезно, когда в feature-ветке есть исправление бага, которое нужно срочно в main, но вся фича ещё не готова.

## 8.5. git tag — метки версий

Тег — это именованная ссылка на коммит. Используется для версионирования релизов:

```
# Лёгкий тег (просто указатель)
git tag v1.0.0

# Аннотированный тег (с сообщением)
git tag -a v1.0.0 -m "Первый релиз"

# Тег на конкретный коммит
git tag -a v0.9.0 a3f2c1d

# Посмотреть все теги
git tag
# v0.9.0
# v1.0.0
# v1.1.0

# Отправить теги на удалённый
git push origin v1.0.0
git push origin --tags  # все
```

### 8.6. Semantic Versioning

Семантическое версионирование (SemVer): формат MAJOR.MINOR.PATCH.

- MAJOR — несовместимые изменения API
- MINOR — новые фичи, обратная совместимость
- PATCH — исправления багов

```
1.0.0  → 1.0.1   # bugfix
1.0.1  → 1.1.0   # новая фича
1.1.0  → 2.0.0   # breaking change
```

### 8.7. git rm и git mv

```
# Удалить файл и закоммитить удаление
git rm old-file.php
git commit -m "chore: удалил старый файл"

# Переименовать/переместить
git mv old.php new.php
git commit -m "refactor: переименовал old.php → new.php"

# Удалить из индекса, но оставить на диске
git rm --cached secret.txt
# (если случайно закоммитили секрет)
```

---

# ГЛАВА 9. КОНВЕНЦИИ И WORKFLOW

## 9.1. Conventional Commits

Стандарт формата коммитов. Помогает читать историю, автоматически генерировать changelog и определять версии для релизов.

Формат: type(scope): description

- feat — новая функциональность
- fix — исправление бага
- docs — документация
- style — форматирование, без логики
- refactor — переработка без изменения поведения
- test — добавление тестов
- chore — обслуживание, зависимости
- perf — улучшение производительности
- build — система сборки, зависимости
- ci — CI/CD конфигурация

```
git commit -m "feat(auth): добавлена регистрация"
git commit -m "fix(forms): валидация email"
git commit -m "docs: обновил README"
git commit -m "refactor(db): вынес PDO в класс Database"
git commit -m "chore: обновил зависимости composer"
git commit -m "perf: индекс для users.email"

# Breaking change (увеличивает MAJOR)
git commit -m "feat(api)!: изменён формат ответа"
```

## 9.2. Git Flow

Классическая модель для проектов с релизным циклом. Ветви:

- main — продакшен, только релизы
- develop — разработка, интеграция фич
- feature/* — отдельные фичи от develop
- release/* — подготовка релиза от develop
- hotfix/* — срочные исправления main

```
# Создать фичу
git switch -c feature/search develop
# ... работаем ...
git switch develop
git merge --no-ff feature/search

# Релиз
git switch -c release/v1.2.0 develop
# ... только багфиксы ...
git switch main
git merge --no-ff release/v1.2.0
git tag -a v1.2.0 -m "Релиз 1.2.0"
git switch develop
git merge --no-ff release/v1.2.0
```

Git Flow избыточен для маленьких проектов. Больше подходит для крупных команд с релизами раз в месяц.

## 9.3. GitHub Flow

Простая модель, используемая GitHub. Только две ветки:

- main — всегда готов к деплою
- feature/* — короткоживущие ветки для фич

Процесс:

```
# 1. Создать ветку от main
git switch -c feature/search

# 2. Делать коммиты
git add . &amp;&amp; git commit -m "feat: поиск"

# 3. Отправить в origin
git push -u origin feature/search

# 4. На GitHub открыть Pull Request
# 5. Code review, обсуждение, правки
# 6. После approve: merge в main
# 7. Удалить feature-ветку
git switch main &amp;&amp; git pull
git branch -d feature/search
```

## 9.4. Trunk-Based Development

Ещё проще: все коммитят прямо в main. Feature-флаги включают неготовый код до релиза. Подходит для опытных команд с CI/CD и короткими циклами.

### 9.5. Workflow для соло

Для одиночной разработки GitHub Flow — оптимален:

```
# Каждый день:
git switch main &amp;&amp; git pull
git switch -c feature/new-thing
# ... кодинг ...
git add . &amp;&amp; git commit -m "feat: ..."
git push -u origin feature/new-thing

# Если фича готова:
git switch main
git merge --no-ff feature/new-thing
git push
git branch -d feature/new-thing

# Деплой: см. главу 10
```

---

# ГЛАВА 10. GIT-ДЕПЛОЙ

### 10.1. Деплой через git pull

Самый простой способ: клонировать репо на сервере и обновлять через git pull. По учебнику Хостинг:

```
# На сервере, один раз
ssh deploy@server
cd /var/www
git clone git@github.com:ivan/mysite.git

# Каждый деплой
ssh deploy@server
cd /var/www/mysite
git pull origin main

# Если есть миграции БД
php migrator.php

# Перезапустить PHP-FPM (если сменилась версия)
sudo systemctl reload php8.3-fpm
```

Плюсы: просто. Минусы: на сервере лежит весь .git с историей, нужен SSH-доступ к GitHub с сервера, нет атомарности переключения.

## 10.2. Bare-репозиторий + hook

Более продвинутый способ: создать на сервере bare-репозиторий (только .git, без рабочей директории) и post-receive hook, который при каждом push обновляет боевой сайт.

Шаг 1. Создаём bare-репозиторий:

```
ssh deploy@server
mkdir -p ~/repos/mysite.git
cd ~/repos/mysite.git
git init --bare

# --bare означает: репо без рабочей директории,
# только .git содержимое. Используется для приёма push.
```

Шаг 2. Создаём post-receive hook:

```
# ~/repos/mysite.git/hooks/post-receive
#!/bin/bash
set -e

TARGET=/var/www/mysite
GIT_DIR=/home/deploy/repos/mysite.git

while read oldrev newrev ref; do
    branch=$(echo $ref | sed 's|refs/heads/||')
    if [ "$branch" == "main" ]; then
        echo "Deploying branch: $branch"
        git --work-tree=$TARGET --git-dir=$GIT_DIR \
            checkout -f $branch
        # Миграции БД
        cd $TARGET &amp;amp;&amp;amp; php migrator.php
        echo "Deploy done."
    fi
done

# Сделать исполняемым
chmod +x ~/repos/mysite.git/hooks/post-receive
```

Шаг 3. На локальной машине добавляем сервер как remote и пушим:

```
# На локальной машине
cd ~/projects/mysite
git remote add deploy \
    deploy@server:~/repos/mysite.git

# Первый push
git push deploy main

# Hook выполнится, файлы появятся в /var/www/mysite
```

### 10.3. Атомарный деплой с символическими ссылками

Проблема предыдущего способа: во время checkout сайт может быть в полуобновлённом состоянии. Решение — атомарное переключение через символическую ссылку.

```
# Структура:
/var/www/
    mysite/        → symlink на releases/NNN
    releases/
        001/       ← каждая версия в своей папке
        002/
        003/       ← текущая (указывает mysite/)
    shared/
        data/database.sqlite  ← общая БД между версиями

# Hook обновляет releases/NNN, потом атомарно
# переключает symlink mysite →
ln -sfn /var/www/releases/003 /var/www/mysite
```

### 10.4. .gitattributes для деплоя

Чтобы не тащить на сервер разработческие файлы (тесты, документацию), используют .gitattributes:

```
# .gitattributes
tests/            export-ignore
docs/             export-ignore
.github/          export-ignore
*.md              export-ignore
.gitignore        export-ignore
.env.example      export-ignore

# При git archive или экспорте эти файлы исключаются
```

---

# ГЛАВА 11. АВАРИЙНОЕ ВОССТАНОВЛЕНИЕ

### 11.1. git reset — откат HEAD

git reset перемещает указатель HEAD на предыдущий коммит. Три режима в зависимости от того, что делать с изменениями в рабочей директории:

- reset --soft: оставить изменения в индексе
- reset --mixed (по умолчанию): в рабочей директории, не в индексе

- reset --hard: всё выбросить, безвозвратно

```
# Отменить последний коммит, изменения оставить
git reset --soft HEAD~1

# Отменить коммит, изменения в рабочей директории
git reset HEAD~1

# Полностью откатить к предыдущему коммиту
# ВНИМАНИЕ: несохранённые изменения потеряются!
git reset --hard HEAD~1
```

## 11.2. git revert — безопасный откат

Вместо переписывания истории, git revert создаёт новый коммит, который отменяет указанный. Безопасно для публичных веток:

```
git revert a3f2c1d

# Создаст коммит "Revert: feat: форма обратной связи"
# с обратными изменениями

# Отменить несколько коммитов
git revert a3f2c1d..b8e4f02

# Отмена без автоматического коммита
git revert --no-commit a3f2c1d
# потом вручную: git commit
```

## 11.3. git reflog — спасение

reflog (reference log) отслеживает все перемещения HEAD, даже те, что были переписаны reset'ом. Это ваш последний шанс восстановить потерянные коммиты.

```
git reflog

# 8b2e9f0 HEAD@{0}: reset: moving to HEAD~1
# a3f2c1d HEAD@{1}: commit: feat: важная фича
# 9c1d7e8 HEAD@{2}: commit: fix: typo
# b8e4f02 HEAD@{3}: checkout: moving from feature to main

# Вернуться к "потерянному" коммиту
git reset --hard a3f2c1d
```

reflog хранится локально, обычно 90 дней. За это время можно восстановить почти всё, что случайно удалили.

## 11.4. Восстановление удалённой ветки

Если случайно удалили ветку — она всё ещё в reflog:

```
git branch -D feature/search  # случайное удаление

# Найти последний коммит этой ветки
git reflog | grep 'feature/search'
# b8e4f02 HEAD@{5}: commit: feat: search — это он

# Восстановить
git switch -b feature/search b8e4f02
```

## 11.5. git bisect — бинарный поиск

Бывает: сайт работал, потом сломался, и непонятно, какой коммит виноват. bisect бинарным поиском находит проблемный коммит за log2(N) шагов.

```
# Запустить bisect
git bisect start

# Указать сломанный коммит (текущий)
git bisect bad

# Указать рабочий коммит (старый)
git bisect good v1.0.0

# Git переключится на середину между ними
# Проверьте, работает ли сайт

git bisect good  # если работает
git bisect bad   # если сломан

# После нескольких итераций Git найдёт коммит:
# b8e4f02 is the first bad commit

# Завершить
git bisect reset
```

Можно автоматизировать:

```
git bisect start HEAD v1.0.0
git bisect run php tests/e2e.php
# Git сам прогонит тесты на каждом коммите
```

## 11.6. Шпаргалка восстановления

- Случайно закоммитил не туда → reset --soft HEAD~1
- Случайно закоммитил в main → reset, потом коммит в ветке

- Случайно сделал reset --hard → git reflog + reset --hard HEAD@{1}
- Случайно удалил ветку → reflog + switch -b имя хэш
- Нужно отменить публичный коммит → git revert
- Сайт сломался после коммита → git bisect
- Случайно закоммитил .env → git rm --cached .env + amend

---

# ГЛАВА 12. GITHUB-ФИЧИ, ЗАДАЧИ И ИТОГИ

## 12.1. Issues — задачи

Issues — встроенный трекер задач. Каждый issue имеет номер, заголовок, описание, метки, ответственного, статус. Можно ссылаться на коммиты и PR.

Шаблон issue через .github/ISSUE_TEMPLATE/:

```
<!-- .github/ISSUE_TEMPLATE/bug_report.md -->
---
name: Bug report
about: Сообщить о баге
---

## Описание бага
Что произошло, что ожидалось.

## Шаги воспроизведения
1. Зайти на ...
2. Нажать ...
3. Видеть ошибку

## Окружение
- ОС: ...
- Браузер: ...
- Версия: ...
```

## 12.2. Pull Requests

PR — запрос на вливание изменений из ветки в main. Это центральный элемент GitHub Flow. PR можно обсуждать, оставлять комментарии к конкретным строкам, требовать review.

- Open PR: из feature/* в main
- Code review: коллеги комментируют
- CI checks: автоматические тесты

- Approve: рецензент подтверждает
- Merge: склеивание (merge/squash/rebase)
- Delete branch: автоматически после merge

Шаблон PR через .github/PULL_REQUEST_TEMPLATE.md:

```
## Что сделано
Краткое описание изменений.

## Зачем
Ссылка на issue: #123
Или описание проблемы.

## Как тестировать
1. Запустить ...
2. Проверить ...

## Чек-лист
- [ ] Код следует стилю проекта
- [ ] Тесты добавлены/обновлены
- [ ] Документация обновлена
```

## 12.3. GitHub Actions — CI/CD

Actions — встроенная система CI/CD. Конфиг в .github/workflows/:

```
# .github/workflows/test.yml
name: Tests
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with:
          php-version: '8.3'
      - run: composer install
      - run: php test.php
```

При каждом push или PR GitHub автоматически запустит тесты на свежей Ubuntu с PHP 8.3. Результат виден прямо в PR.

## 12.4. GitHub Pages — бесплатный хостинг

GitHub Pages — бесплатный хостинг для статичных сайтов. Репозиторий становится сайтом по адресу username.github.io/repo.

```
# Включить: Settings → Pages
# Source: branch main / (root)
# Сайт появится на:
# https://ivan.github.io/mysite/

# Своё доменное имя: в DNS добавить CNAME
# mysite.ru → ivan.github.io
#□□□ репо добавить файл CNAME:
echo "mysite.ru" > CNAME
git add CNAME &amp;amp;&amp;amp; git commit -m "chore: CNAME"
git push
```

## 12.5. Wiki

GitHub Wiki — встроенный wiki для документации проекта. Подходит для руководств, архитектурных документов, FAQ. Markdown, боковое меню, ревизии.

## 12.6. .github/ — специальные файлы

```
.github/
    ISSUE_TEMPLATE/
        bug_report.md
        feature_request.md
    workflows/
        test.yml       # CI
        deploy.yml     # CD
    PULL_REQUEST_TEMPLATE.md
    CODEOWNERS         # кто отвечает за папки
    FUNDING.yml        # спонсоры
    dependabot.yml     # автообновление зависимостей
```

---

## 12.7. Задача 1: первый коммит

Создайте репозиторий, добавьте README.md, сделайте первый коммит.
Решение:

```
mkdir hello-git &amp;amp;&amp;amp; cd hello-git
git init
git config user.name "Ваше Имя"
git config user.email "you@example.com"

echo "# Hello Git" > README.md
git add README.md
git commit -m "feat: первый коммит"

git log --oneline
```

## 12.8. Задача 2: ветка и merge

Создайте ветку feature/greeting, добавьте файл greeting.txt, влейте в main. Решение:

```
git switch -c feature/greeting
echo "Hello, World!" > greeting.txt
git add greeting.txt
git commit -m "feat: добавлен greeting"

git switch main
git merge feature/greeting
git branch -d feature/greeting

git log --oneline --graph
```

## 12.9. Задача 3: разрешение конфликта

Внесите изменения в одну строку README.md в обеих ветках, чтобы вызвать конфликт. Разрешите его. Решение:

```
# В main
echo "# My Project" > README.md
git commit -am "docs: переименовал"

# В feature/x (до pull)
git switch -c feature/x
echo "# Hello Project" > README.md
git commit -am "docs: привет"

# Слияние с конфликтом
git switch main
git merge feature/x
# CONFLICT в README.md

# Открываем README.md, видим:
# <<<<<<< HEAD
# # My Project
# =======
# # Hello Project
# >>>>>>> feature/x

# Редактируем вручную:
# # Hello Project

git add README.md
git commit  # с сообщением "Merge ..."
```

## 12.10. Задача 4: .gitignore для PHP-проекта

Создайте .gitignore для PHP+SQLite проекта с секретами. Решение:

```
# .gitignore

# Секреты
.env
.env.*
!.env.example

# БД SQLite
data/*.sqlite
data/*.sqlite-*

# PHP
vendor/
composer.lock

# Логи
*.log
/tmp/

# IDE
.idea/
.vscode/
*.swp

# ОС
.DS_Store
Thumbs.db

# Загруженные пользователями
public/uploads/

# Сделать первый коммит .gitignore
git add .gitignore
git commit -m "chore: добавлен .gitignore"
```

## 12.11. Задача 5: Git-деплой

Настройте деплой сайта через git push на сервер. Решение:

```
# НА СЕРВЕРЕ:
ssh deploy@server
mkdir -p ~/repos/mysite.git
cd ~/repos/mysite.git
git init --bare

cat > hooks/post-receive << 'EOF'
#!/bin/bash
set -e
TARGET=/var/www/mysite
GIT_DIR=$PWD
while read oldrev newrev ref; do
    branch=${ref#refs/heads/}
    if [ "$branch" = "main" ]; then
        git --work-tree=$TARGET --git-dir=$GIT_DIR \
            checkout -f $branch
        cd $TARGET &amp;amp;&amp;amp; php migrator.php
        echo "[$(date)] Deploy OK" >> /tmp/deploy.log
    fi
done
EOF
chmod +x hooks/post-receive

# НА ЛОКАЛЬНОЙ МАШИНЕ:
cd ~/projects/mysite
git remote add deploy deploy@server:~/repos/mysite.git
git push deploy main

# Hook выполнится, сайт обновится автоматически.
```

## 12.12. Задача 6: восстановление через reflog

Случайно сделали git reset --hard HEAD~3 и потеряли три коммита. Восстановите. Решение:

```
git reflog

# 8b2e9f0 HEAD@{0}: reset: moving to HEAD~3
# b8e4f02 HEAD@{1}: commit: feat: третья фича
# 9c1d7e8 HEAD@{2}: commit: feat: вторая фича
# a3f2c1d HEAD@{3}: commit: feat: первая фича
# ... остальное ...

# Восстановить к HEAD@{1} (до reset):
git reset --hard HEAD@{1}

# Проверить
git log --oneline
# b8e4f02 (HEAD -> main) feat: третья фича
# 9c1d7e8 feat: вторая фича
# a3f2c1d feat: первая фича
```

## 12.13. Задача 7: GitHub Pages

Разверните статический сайт на GitHub Pages. Решение:

```
# 1. Создать репо на GitHub: ivan/mysite
# 2. Локально:
mkdir mysite &amp;&amp; cd mysite
git init
echo '<h1>Hello!</h1>' > index.html
git add . &amp;&amp; git commit -m "feat: index"
git remote add origin \
    git@github.com:ivan/mysite.git
git push -u origin main

# 3. На GitHub:
#    Settings → Pages → Source: main / (root)
#    Save

# 4. Через 1-2 минуты сайт доступен:
#    https://ivan.github.io/mysite/

# 5. Свой домен:
echo "mysite.ru" > CNAME
git add CNAME
git commit -m "chore: CNAME для домена"
git push

# 6. DNS: CNAME mysite.ru → ivan.github.io
```

---

## 12.14. Итоги по главам

Глава 1. Git — распределённая система контроля версий. Установка и первая настройка: user.name/email, init.defaultBranch=main.

Глава 2. Базовый цикл: init → status → add → commit → log. Четыре состояния файла: untracked/staged/committed/modified.

Глава 3. git log (опции --oneline/--graph/-p), git show, git diff (рабочая vs индекс vs HEAD). Чтение diff: +/-, @@, имена файлов.

Глава 4. Ветки: branch, switch -c, switch, branch -d/-D. Конвенция наименования: feature/, fix/, hotfix/, release/.

Глава 5. git merge, fast-forward vs --no-ff. Конфликты: маркеры <<<<<<< HEAD ======= >>>>>>> branch. Разрешение вручную, --ours/--theirs, --abort.

Глава 6. rebase: пересадка коммитов на актуальный main. Интерактивный rebase -i: squash/reword/fixup/drop. Золотое правило: не rebase публичные ветки. Reflog как страховка.

Глава 7. Удалённые: GitHub/GitLab, clone, remote add, push, pull, fetch. SSH-ключи для бесшовной авторизации. Fork и upstream для контрибуции.

Глава 8. .gitignore (шаблон для PHP+SQLite), stash (отложить изменения), cherry-pick (перенос коммита), tag (SemVer: MAJOR.MINOR.PATCH), rm/mv.

Глава 9. Conventional Commits (feat/fix/docs/refactor/...). Git Flow (5 веток), GitHub Flow (main + feature PR), Trunk-Based. Соло: GitHub Flow оптимален.

Глава 10. Git-деплой: простой git pull на сервере, или bare-репо + post-receive hook для автоматического обновления при push. Атомарный деплой через символические ссылки.

Глава 11. Аварийное: reset --soft/--mixed/--hard, revert (безопасно для публичных), reflog (восстановление переписанной истории), bisect (бинарный поиск сломавшего коммита).

Глава 12. GitHub: Issues (шаблоны), Pull Requests (review, CI), Actions (CI/CD), Pages (бесплатный хостинг), Wiki, .github/ папка. Семь практических задач.

---

## 12.15. Что читать дальше

После освоения этого учебника логично двигаться в трёх направлениях.
Во-первых, Pro Git book (git-scm.com/book/ru) — бесплатная книга от авторов Git, исчерпывающее руководство.

Во-вторых, GitHub Docs (docs.github.com) — детальная документация по Actions, Pages, организациям, проектам, CLI gh. GitHub CLI позволяет создавать PR, issues, релизы прямо из терминала.

В-третьих, продвинутые темы: git submodules (вложенные репозитории), git-lfs (большие файлы), git worktree (несколько рабочих директорий одного репо), кастомные hooks через husky.

Главный совет: не пытайтесь заучить все команды. Освойте базовый цикл (init/add/commit/push/pull), потом постепенно добавляйте ветки, rebase, hooks. Через полгода ежедневной работы с Git команды уходят в мышечную память, а сложные случаи (конфликты, reflog) решаются по документации. Опыт складывается из десятков сделанных и переделанных коммитов, а не из прочитанных справочников.

---